

Informatics II, Spring 2026, Exercise 3

Publication of exercise: March 9, 2026

Publication of solution: March 16, 2026

Exercise classes: March 16 - March 20, 2026

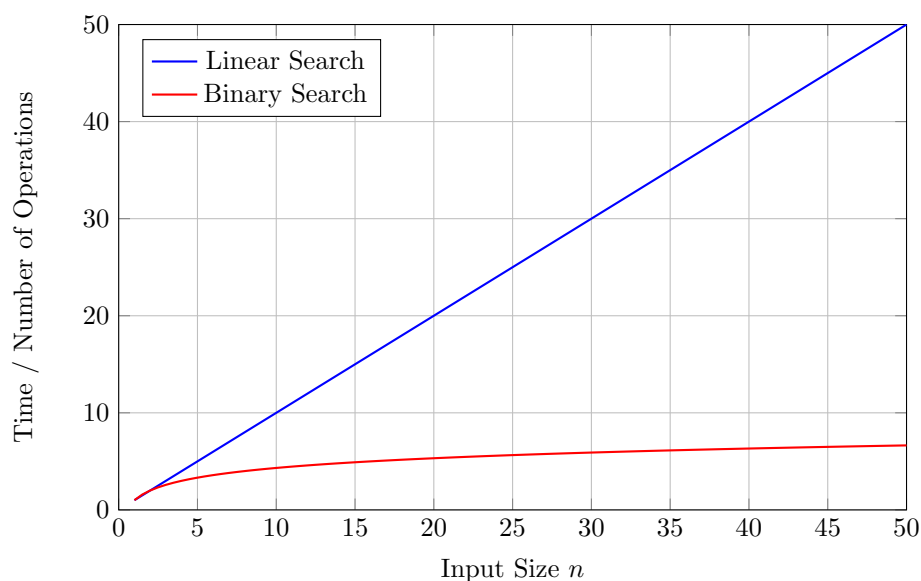
Task 1 (Easy)

Consider an array A with n distinct integers that are sorted in an ascending order and an integer t .

a) The C function `linear_search` traverses the integers in A , one after another, from the beginning. If t is found in A `linear_search` returns 1, otherwise 0. Referencing the pseudocode for binary search in SL02, implement the C function `binary_search(int A[], int n, int t)` that employs binary search to find integer t in A . If t is found in A `binary_search` returns 1, otherwise 0. Complete the C function `binary_search(int A[], int n, int t)` in `task1.c` file.

[An example solution is provided in `task1solution.c`.](#)

b) Assuming the plot below displays the average runtime for Linear and Binary Search. Explain the differences in runtime. What is the worst case complexity of both algorithms?



Runtime comparison of linear search and binary search as a function of input size n . Please note that the displayed values have been mocked and where not measured.

Run time for linear search grows linearly when n grows ($O(n)$).

Run time for binary search grow logarithmically when n grows ($O(\log(n))$).

For binary search, in each iteration i , we can discard half of the elements which were left to be checked before iteration i .

c) Run your implementation of `binary_search` against the `linear_search` using the provided `compare_binary_and_linear()`. Adjust the parameters as desired. Does your implementation follow the same behavior as in the figure above?

While the absolute value differ from machine to machine, the general trend that binary search is faster, is always the case.

Task 2 (Medium)

In this task you are given multiple code snippets to analyse in regards to the runtime. Feel free to reuse previously calculated results in future tasks of this exercise file.

a) For each of the following loops decide, how many times each line of code is executed.

i)

```
for i = n to 1 by -1 do
  print("x")
```

ii)

```
for i = 0 to n by 3 do
  print("x")
```

iii)

```
for i = 0 to n do
  for j = 0 to m do
    print("x")
```

Note: In the above pseudocode, `for i = 0 to n by k` means we do not increase i by 1, but each time, we increase it by k , i.e., $i=i+k$.

i) Recursive formulation: $i_{k+1} = i_k - 1$, $i_0 = n$, we can rewrite this explicitly into $i_{k+1} = i_0 - k \cdot 1 = n - k$. Thus for the inner loop will run until $i_{k+1} = 1 \Rightarrow 1 = n - k \Rightarrow k = n - 1$. In words we can say that we can subtract $n - 1$ times 1 from n to arrive at 1. However, the loop will also run for $i = n$ once. So in total the content of the loop will be executed n times. The loop condition will be verified once more, namely if $i = 0$ is larger or equal to 1.

For loop statement: $n + 1$ times

print statement: n times

ii) In each iteration $k + 1$ we add 3 to the value i_k of iteration k ($i_{k+1} = i_k + 3$, $i_0 = 0$). We can rewrite this explicitly as $i_{k+1} = i_0 + 3 \cdot k$ and find for $i_{k+1} = n \geq 0 + 3 \cdot k = i_0 + 3 \cdot k \Rightarrow n/3 \geq k$. Thus we can follow that the loop's content will be executed $n/3 + 1$ times and the loop condition once more for a value larger than n .

For loop statement: $n/3 + 2$ times

print statement: $n/3 + 1$ times

iii) For the outer loop we can use the same argument as in i) with $+1$: $i_{k+1} = i_k + 1$, $i_0 = 0 \Rightarrow i_{k+1} = 0 + k \cdot 1 = k \Rightarrow n \leq k \Rightarrow k = n$. We can add n times 1 to the initial value 0 and for each value the loop's content will be executed: $n + 1$. For the loop statement, it's condition will be checked once more for $i_{k+1} = n + 1 > n$ resulting in $n + 2$ times.

Outer loop statement: $n + 2$

Outer loop content: $n + 1$

The inner loop's condition is independent of the values of the outer loop: j is independent of i . Thus for each iteration of i of the outer loop, the same number of iterations for the inner loop will be performed. The number of iterations for the inner loop can be calculated in the same way as above:

Inner loop statement: $m + 2$

Inner loop content: $m + 1$

In total we get the following values:

Line 1: Outer loop statement: $n + 2$ times

Line 2: Inner loop statement: $(n + 1) \cdot (m + 2) = nm + 2n + m + 2$ times

Line 3: Print statement: $(n + 1) \cdot (m + 1) = nm + n + m + 1$ times

b) Below is a pseudocode of a function named `whatDoesItDo`, which takes an array $A[1..n]$ of n integers and an integer k as inputs. Feel free to use intermediate results from **a)**.

```

Algo: whatDoesItDo(A, n, k)
result = -1000
for i = 1 to n do
    current = 0
    for j = i to n by k do
        current = current + A[j]
    if current > result then
        result = current
return result

```

- Perform exact analysis of the running time of the algorithm.

Instruction	# of times executed	Cost
result = -1000	1	c_1
for i := 1 to n do	$n + 1$	c_2
current = 0	n	c_3
for j := i to n by k do	$\frac{n^2-n}{2k} + 2n$ *	c_4
current = current + A[j]	$\frac{n^2-n}{2k} + n$ **	c_5
if current > result then	n	c_6
result = current	αn ***	c_7
return result	1	c_8

$$* (1 + \frac{(n-1)}{k} + 1) + (1 + \frac{(n-2)}{k} + 1) + (1 + \frac{(n-3)}{k} + 1) + \dots + (1 + \frac{(n-i)}{k} + 1) + \dots + (1 + \frac{(n-n)}{k} + 1) = \frac{n^2-n}{2k} + 2n$$

$$** (1 + \frac{(n-1)}{k}) + (1 + \frac{(n-2)}{k}) + (1 + \frac{(n-3)}{k}) + \dots + (1 + \frac{(n-i)}{k}) + \dots + (1 + \frac{(n-n)}{k}) = \frac{n^2-n}{2k} + n$$

$$*** \alpha \in [0, 1]$$

$$T(n) = c_1 + c_2(n + 1) + c_3n + c_4(\frac{n(n+1)}{2k}) + c_5(\frac{n(n+1)}{2k} - 1) + c_6(n) + c_7(\alpha n) + c_8$$

Detailed explanation for ** (same is applicable for *):

In this task, the inner and outer loop are coupled, the inner loop's condition depends on the current value of the outer loop. The outer loop's content will be executed $S_o = n$ times. For the inner loop, for a certain i we get $\frac{n-i}{k} + 1$ iterations:

Value for i	Inner loop condition	Number of iterations
$i = 1$	for $j := 1$ to n do	$\frac{n-1}{k} + 1$
$i = 2$	for $j := 2$ to n do	$\frac{n-2}{k} + 1$
...	...	...
$i = l$	for $j := l$ to n do	$\frac{n-l}{k} + 1$
...	...	...
$i = n$	for $j := n$ to n do	$\frac{n-n}{k} + 1$

To calculate the total number of iterations over both loops, we can use a mathematically trick to sum up all entries in the table above. We combine the first with the last entry, 2nd and 2nd last and so on to create pairs with the same combined value:

$$\begin{aligned}
 & \left(1 + \frac{(n-1)}{k}\right) + \left(1 + \frac{(n-2)}{k}\right) + \left(1 + \frac{(n-3)}{k}\right) + \dots + \left(1 + \frac{(n-i)}{k}\right) + \dots + \left(1 + \frac{(n-n)}{k}\right) \\
 &= \left[\left(1 + \frac{(n-1)}{k}\right) + \left(1 + \frac{(n-n)}{k}\right) \right] + \left[\left(1 + \frac{(n-2)}{k}\right) + \left(1 + \frac{(n-(n-1))}{k}\right) \right] + \dots \\
 &= \left[2 + \frac{n-1}{k} \right] + \left[2 + \frac{n-1}{k} \right] + \dots \\
 &= \left[2 + \frac{n-1}{k} \right] \cdot \frac{n}{2} = n + \frac{n^2 - n}{2k}
 \end{aligned}$$

- Determine the asymptotic complexity of the algorithm?

As n gets larger, the leading term in the above formula is n^2 . Therefore, the asymptotic complexity of the algorithm is $O(n^2)$.

Task 3 (Easy)

Calculate the asymptotic tight bound for the following functions and rank them by their order of growth (lowest first). Clearly work out the calculation step by step in your solution.

$$f_1(n) = 10^{\lg 20} n^4 + 8^{229} n^3 + 20^{231} n^2 + 128n \log n$$

$$f_2(n) = \log^2(n) + 50\sqrt{n} + \log(n)$$

$$f_3(n) = 14400$$

$$f_4(n) = 2 \log(6^{\log n^2}) + \log(\pi n^2) + 4^{\log_2 n}$$

$$f_5(n) = (2n + 3)!$$

$$\begin{aligned} f_1(n) &= 10^{18} 20 n^4 + 8^{229} n^3 + 20^{231} n^2 + 128 n \log n \\ &= c_1 \cdot n^4 + c_2 \cdot n^3 + c_3 \cdot n^2 + c_4 \cdot n \log n, \quad c_1, c_2, c_3, c_4 \in \mathbb{R} \\ &= \Theta(n^4) \end{aligned}$$

$$\begin{aligned} f_2(n) &= \log^2(n) + 50\sqrt{n} + \log n \\ &= c_1 \cdot \log^2(n) + c_2 \cdot \sqrt{n} + c_3 \cdot \log n, \quad c_1, c_2, c_3 \in \mathbb{R} \\ &= \Theta(\sqrt{n}) \end{aligned}$$

$$\begin{aligned} f_3(n) &= 14400 \\ &= \Theta(1) \end{aligned}$$

$$\begin{aligned} f_4(n) &= 2 \log(6^{\log n^2}) + \log(\pi n^2) + 4^{\log_2 n} \\ &= 2 \log(6) \cdot (\log n^2) + \log(\pi n^2) + (2^2)^{\log_2 n} \\ &= 2 \log(6) \cdot (2 \log(n)) + \log(\pi) + \log(n^2) + 2^{2 \cdot (\log_2 n)} \\ &= 2 \log(6) \cdot 2 \log(n) + \log(\pi) + 2 \log(n) + \left(2^{\log_2(n)}\right)^2 \\ &= 2 \log(6) \cdot 2 \log(n) + \log(\pi) + 2 \log(n) + n^2 \\ &= \Theta(n^2) \end{aligned}$$

$$\begin{aligned} f_5(n) &= (2n+3)! \\ &= (2n)! \cdot (2n+1) \cdot (2n+2) \cdot (2n+3) \\ &= \Theta((2n+3)!) \\ &\neq \Theta((2n)!) \\ &\neq \Theta(n!) \end{aligned}$$

$$\Theta(f_3(n)) < \Theta(f_2(n)) < \Theta(f_4(n)) < \Theta(f_1(n)) < \Theta(f_5(n))$$

Task 4 (Hard)

Consider the algorithm `algo1`. The input parameters are an array `A[1..n]` with n distinct integers and $k \leq n$.

```

Algo: algo1(A, n, k)

sum = 0;
for i = 1 to k do
    maxi = i;
    for j = i to n do
        if A[j] > A[maxi] then
            maxi = j;
    sum = sum + A[maxi];
    swp = A[i];
    A[i] = A[maxi];
    A[maxi] = swp;
return sum

```

- a) In your own words, describe what the algorithm **algo1** does? How do the inner and outer loop contribute to achieving this goal?

The algorithm returns the sum of the first k largest elements in the array. It will also sort the first k elements in descending order. The inner loop is responsible for finding the i -th largest elements in the sub-array $[i, \dots, n]$. The outer loop runs k times in order to find the largest elements in the sub-array k times.

- b) Based on your description above, formalize the pre/post conditions of the **algo1** algorithm.

The preconditions (inputs) are an array $A[1..n]$ and an integer k .

The post conditions(outputs) are the following:

- sum of the biggest k elements of the array $A[1, \dots, n]$. Recursively, we can define the output of **algo1** (sum of the biggest k elements of the array $A[1, \dots, n]$) in the following way: Let $sum \in \mathbb{N}$ denote the biggest k elements of the array $A[1, \dots, n]$, then we have

$\forall i \in [1, \dots, k] : sum = sum + A[i]$, where $A[1, \dots, k]$ are the biggest k integers and sorted in a descending order

- Integers of $A[1, \dots, k]$ are the biggest k integers in A and sorted in a descending order.

- c) For the two **for** loops in the algorithm:

- i. Determine if the loop is **up** loop or **down** loop.

The outer loop **for** $i = 1$ **to** k is an **up** loop, as it runs from low (1) to high k .

The inner loop **for** $j = i$ **to** n is an **up** loop as well, as it runs from low (i) to high n .

- ii. Determine the invariants of these two loops and verify whether they hold in three stages: **initialization**, **maintenance** and **termination**.

We start with the inner loop. The invariant of the inner loop is that $\forall k \in [i, \dots, j] : A[maxi] \geq A[k]$, i.e., $A[maxi]$ is the largest element in the array $A[i, \dots, j]$.

- Initialization.** This invariant holds. At this time, $j = i$ and $A[i, j]$ contains only one element $A[i]$. At this moment (before the loop starts), we have $j = i$ and $A[maxi] = A[i]$ is the only, and the biggest element in $A[i, j] = A[i, i]$.
- Maintenance.** This invariant holds. Before loop $j > i$ starts it must hold that $A[maxi] \geq A[l], \forall l \in [i, \dots, j-1]$. In the loop j if $A[j] > A[maxi]$ we set $maxi = j$, and thus after loop j terminates, independent of if the value of $A[j]$, it holds that $A[maxi] \geq A[l] \forall l \in [i, \dots, j]$.

- iii. **Termination.** The inner loop terminates when $j = n$. At this time, if $A[j] > A[*maxi*]$, then *maxi* is assigned to be n . It allows us to conclude that $A[*maxi*] \geq A[m], \forall m \in [i..j]$. Furthermore, we also have $A[*maxi*] \geq A[m], \forall m \in [i, \dots, n]$ after the loop terminates. In other words, $A[*maxi*]$ is the biggest element in $A[i, \dots, n]$.

Then we analyze the outer loop, **for** $i = 1$ **to** k . The invariant of the outer loop is that $A[1, \dots, i]$ is sorted in descending order and contains the biggest i elements of the array $A[1, \dots, n]$ after iteration i . Thus before iteration i , it must hold that $A[1, \dots, i - 1]$ is sorted in descending order and contains the biggest $i - 1$ elements of the array A .

- i. **Initialization.** Before the first iteration $i = 1$ it must hold that $A[1, i - 1] = A[1, 0] =$ is sorted and contains the $i - 1 = 1 - 1 = 0$ biggest elements. This is true as $A[1, 0] =$ does not contain any elements and thus also cannot be sorted.
- ii. **Maintenance.** $i > 1$ Before iteration i it holds that $\forall p \in [1, \dots, i - 1], \forall q \in [i, \dots, n] : A[p] \geq A[q]$. In iteration i the inner loop will find the biggest element in the sub array $A[i, n]$ and save it's index as *maxi*. We increase the sum by $A[*maxi*]$ and swap the elements at index i with *maxi*, resulting in the i biggest element being at index i . Thus it holds that after iteration i , $A[1, \dots, i]$ is sorted in descending order ($A[1, \dots, i - 1]$ was sorted already and $A[i]$ must be smaller than all elements in $A[1, \dots, i - 1]$, otherwise it would have been selected in an earlier iteration). It also hold that $A[i] > A[q], \forall q \in [i + 1, \dots, n]$, thus we have that after iteration i it holds that $\forall p \in [1, \dots, i], \forall q \in [i + 1, \dots, n] : A[p] \geq A[q]$.
- iii. **Termination.** The loop terminates when $i = k$. $A[1..k]$ is sorted in descending order and contains the largest k elements of $A[1..n]$.

d) Identify some edges cases of the algorithm and verify if the algorithm has the correct output.

- If $A[1..n]$ is empty, then the algorithm only initialize **sum** to be zero and returns it.
- If $A[1..n]$ only contains one element, the outer loop will be executed only once and guarantees the $A[1..n]$ contains the biggest element, which is the only element in the array. The algorithm returns the initialized sum (0) plus the only element in the array ($A[1]$).
- For a general case, the outer loop guarantees that $A[1..n]$ contains the biggest k elements. In the body of the outer loop, the algorithm calculates the sum of the first k elements in the array $A[1..n]$ and returns it. The returned value is the sum of the biggest k elements.

e) Conduct an exact analysis of the running time of algorithm **algo1**.

Instruction	# of times executed	Cost
$sum := 0$	1	c_1
for $i := 1$ to k do	$k + 1$	c_2
$maxi := i$	k	c_3
for $j := i$ to n do	$\left(kn - \frac{k(k-1)}{2}\right)^* + k^{**}$	c_4
if $A[j] > A[maxi]$ then	$kn - \frac{k(k-1)}{2}$	c_5
$maxi := j$	$\alpha \left(kn - \frac{k(k-1)}{2}\right)^{***}$	c_6
$sum := sum + A[maxi]$	k	c_7
$swp := A[i]$	k	c_8
$A[i] := A[maxi]$	k	c_9
$A[maxi] := swp$	k	c_{10}
return sum	1	c_{11}

* $(n) + (n - 1) + \dots + (n - (k - 1)) = kn - (0 + 1 + \dots + k - 1) = kn - \frac{k(k-1)}{2}$

** k times for terminating loops

*** $0 \leq \alpha \leq 1$

$$T(n) = c_1 + c_2(k+1) + c_3k + c_4 \left(kn - \frac{k(k-1)}{2} + k \right) \\ + c_5 \left(kn - \frac{k(k-1)}{2} \right) + c_6 \left(\alpha \left(kn - \frac{k(k-1)}{2} \right) \right) \\ + (c_7 + c_8 + c_9 + c_{10})k + c_{11}$$

- f) Determine the best and the worst case of the algorithm. What is the running time and asymptotic complexity in each case?

Best case

In the best case, the array has already been sorted in descending order, hence we do not need to run `maxi := j`, i.e., $\alpha = 0$. In this case,

$$T_{\text{best}}(n) = c_1 + c_2(k+1) + c_3k + c_4 \left(kn - \frac{k(k-1)}{2} + k \right) \\ + c_5 \left(kn - \frac{k(k-1)}{2} \right) + 0 + (c_7 + c_8 + c_9 + c_{10})k + c_{11} \\ T_{\text{best}}(n) = O(k * n)$$

Worst case

Similarly, in the worst case, the array is sorted in ascending order and we have to update `maxi` every time, i.e., $\alpha = 1$. In this case,

$$T_{\text{worst}}(n) = c_1 + c_2(k+1) + c_3k + c_4 \left(kn - \frac{k(k-1)}{2} + k \right) \\ + c_5 \left(kn - \frac{k(k-1)}{2} \right) + c_6 \left(kn - \frac{k(k-1)}{2} \right) \\ + (c_7 + c_8 + c_9 + c_{10})k + c_{11} \\ T_{\text{worst}}(n) = O(k * n)$$

Asymptotic complexity of best and worst case

$$T_{\text{best}}(n) = O(k * n)$$

$$T_{\text{worst}}(n) = O(k * n)$$

Task 5 (Easy)

For each of the statements below determine if they are True or False with explanation:

(1)

Assume $f(n)$ describes the runtime of an algorithm, then $g(n)$ is an asymptotic upper bound for $f(n)$.

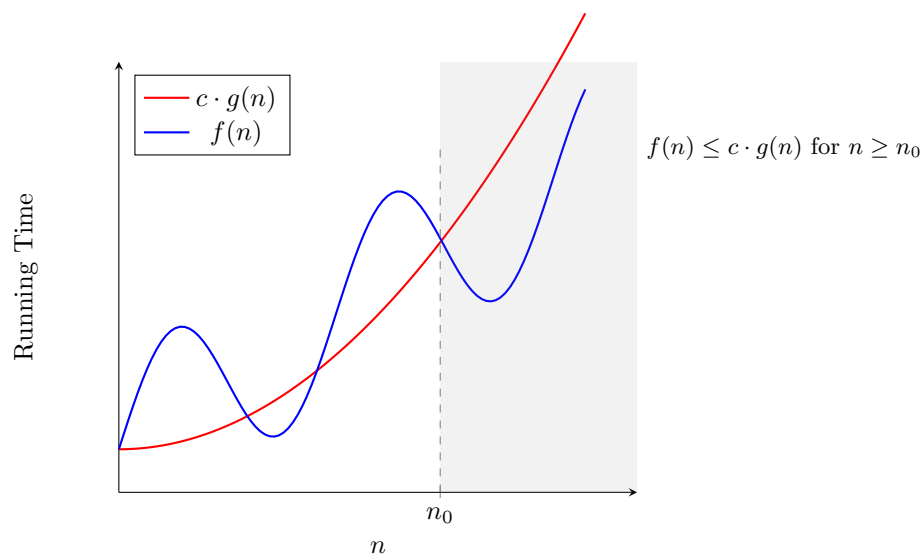


Figure 1: Task 5 (1)

(2)

Each algorithm has at most one upper bound for its running time.

(3)

If an algorithm has a tight bound it always has the same function as upper and as lower bound.

(4)

The content of a loop with condition **for** $int\ i = n; i \geq 1; i = i/2$ runs asymptotically $\log_2(n)$ times.

1. $g(n)$ follows the definition of an upper bound: for a given constant $c \in \mathbb{R}$ and a $n_0 \in \mathbb{N}$ it holds that $f(n) \leq c \cdot g(n), \forall n \geq n_0$

2. False, an algorithm can have multiple upper bounds. For example $h(n) = n^2$ than $n^2 \in \Theta(n^2), n^3 \in \Theta(n^3), \dots$
3. True, if an algorithm with runtime $f(n)$ has a tight bound $g(n)$, then it must hold that $g(n)$ is an upper as well as a lower bound for $f(n)$ (for some constants $c_1, c_2 \in \mathbb{R}$ it holds that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \quad \forall n \geq n_0$).
4. True, in each iteration we divide the previous i through 2: $i_{k+1} = i_k/2, i_0 = n$. Explicitly, we can say that $i_{k+1} = n/2^k$. For $1 = i_{k+1} = n/2^k \Rightarrow 2^k = n \Rightarrow k = \log_2(n)$ we find that the loop's content will be executed $\log_2(n) + 1$ times which is $\Theta(\log_2(n))$